

MERN stack

REACT JS:-

- It is a javascript library
- It was introduced by facebook

Two ways to create react app:-

- 1) `npm create-react-app app-name`
- 2) `npm create vite@latest app-name`

Why react?-

* Faster performance

* lightweight

* Easy code base

* Huge community support

Steps to create React app:-

- 1) Install Node.js
- 2) Desktop → create one folder
- 3) open in vs code
- 4) open terminal
- 5) Give command to create react app

React-JS:- `npm create-react-app app-name`

→ node modules

react, reactrouter Dom, react script, templates

→ all the modules will install

→ slow to install and become slow to use

`npm start`

React-vite:- `npm create vite@latest` → Project name
Package name
Select framework
select a variant

`cd vite-project`

`npm i`

`npm run dev`

Index.html → first file

2

main.jsx

↓

App. 1's X

render:- Whatever ~~do~~ you ~~have~~ ~~writing~~ inside render that will get appended on screen

Strictmode:-

- 1) handle minor errors
- 2) run your application twice.

App.js :- `<App>`

Components:- $\Rightarrow$ First letter will be capital
 $\Rightarrow$ Extension $\rightarrow$.jsx

- 1) class based Component
- 2) functional Component

1) functional component

Functional Component :-

- (1) Javascript function → The function can be named or arrow function. Inside this function we will be writing our code ~~inside~~ in the return.

(2) return statement :-

return Statement:-
The code which should be rendered to display on screen written in this.

- (3) export default function-name
multiple functions ^{export} { }

(-) Statement management done by hooks

(4) Shortcut:- Ratce

Props:-

- Whenever we want to send the data from parent to child component we use props
- passing data is unidirectional (from parent to child)
- Data can be sent:- text, number, functions, images

How to send the data?

text, number, functions, images

How to receive the data

Props, destructuring

Default Props

Default Props → When we fail to send some data at that time default props will append on the screen

Ex:- child

{name="Pooja", age}

Parent

age={21} />

↓

gf i missed name then the default value append on the screen

useState() Hook:-

This hook is used to manage state in functional component

It provides a way to declare state variables and update them, triggering re-renders of the component when the state changes

Syntax:- import {useState} from 'react';

```
const [count, setCount] = useState(0);
```

```
function increment() {
```

```
  setCount(count+1)
```

```
}
```

```
  return C
```

```
}
```


Class based State Management

→ A class component in React is a way to define a component using ES6 classes. Before the introduction of hooks in React 16.8, class components were the primary way to manage state and lifecycle methods within a component.

→ class component extends from `React.Component` and must implement a `render()` method that returns JSX.

```
import React, { Component } from 'react'
```

```
export default class classComponent extends Component {
```

```
  constructor(props) { super(props);
```

```
    this.state = { count: 0; }
```

```
  }
```

```
  increment = () => {
```

```
    this.setState({ count: this.state.count + 1 })
```

```
  };
```

```
  decrement = () => {
```

```
    this.setState({ count: this.state.count - 1 })
```

```
  };
```

```
  render() {
```

```
    return (
```

```
      <div> <h1> Count: { this.state.count } </h1>
```

```
      <button onClick={this.increment}> Increment </button>
```

```
      <button onClick={this.decrement}> Decrement </button>
```

```
    </div>
```

```
  )
```

```
  }
```

```
}
```


How to add CSS in react App?

There are 3 ways to add CSS

- (1) Internal CSS
- (2) External CSS
- (3) Modular CSS

① Internal or Inline Styles

Directly embedded within the component.

Ex:-

```
import React from 'react';

function MyComponent() {
  const myStyle = {
    color: 'blue',
    fontSize: '18px',
  };

  return (
    <div style={myStyle}>
      This text is styled inline
    </div>
  );
}

export default MyComponent;
```

② External CSS:

External stylesheets are imported and applied using CSS selectors.

Ex.- Create a CSS file (myStyle.css):

```
.my-div {
  color: green;
  font-size: 20px;
}
```

Import it into your React Component

```
import React from 'react';
import './myStyle.css';
```



```
function MyComponent() {
```

```
  return (
```

```
    <div className="my-div">
```

```
      This text is styled from external stylesheet
```

```
    </div>
```

```
  )
```

```
}
```

```
export default MyComponent;
```

③ CSS modules :- Scoped styles for individual components, preventing name collisions
Create a CSS module file (MyComponent.module.css)

```
.my-div {
```

```
  color: orange;
```

```
  font-size: 24px;
```

```
}
```

Import it into your React component

```
import React from 'react';
```

```
import styles from './MyComponent.module.css'
```

```
function MyComponent() {
```

```
  return (
```

```
    <div className={styles.my-div}>
```

```
      This text is styled from a CSS  
      module
```

```
    </div>
```

```
  )
```

```
}
```

```
export default MyComponent
```

Also, we can use styled components package
that is CSS-in-JS library to write styles in
Javascript

Lists and Keys

List:- Ordered List

Unordered list

Keys:- A unique entity attached to List or data

Ex:- import React from 'react' (Fruitlist.jsx)
const Fruitlist = () => {
 const Fruits = ["apple", "banana", "mongo"]
 return (
 <>

 {
 Fruits.map((item) => ({item}))
 }

 </>
)
}

export default Fruitlist

This code give warning in console
Each child in a list should have a unique "key"
prop

So, give key attribute to the list
Fruits.map((item, index) =>
<li key={index}> {item}

Realworld example:-

We use Keys for Timestamp

Usecase of Keys:-

For fetching API data

- List rendering → with the help of `Array.map()` or `Array.forEach()`
- Visualization
- State updates :- Keys enable React to accurately determine which components need to be updated when the state changes

Creation of table in React

Table.jsx:-

```
import React from 'react'
```

```
const Table = () => {
```

```
  const Students = [{ id: 1, name: "A", course: "Javascript" },  
                    { id: 2, name: "B", course: "Python" }  
  ]
```

```
  return (
```

```
    <>
```

```
      <table>
```

```
        <thead>
```

```
          <tr>
```

```
            <th>ID </th>
```

```
            <th>NAME </th>
```

```
            <th>COURSE </th>
```

```
          </tr>
```

```
        </thead>
```

```
        <tbody>
```

```
          {StudentData.map((item) => {  
            <tr key={item.id}>  
              <td>{item.name}</td>  
              <td>{item.course}</td>
```

```
            </tr>
```

```
          )} </tbody>
```

```
        </table>
```

```
      </>
```

```
    )
```

```
  export default Table
```


Usecase of Keys:-

For fetching API data

- List rendering → with the help of `Array.map()` or `Array.forEach()`
- Visualization
- State updates :- Keys enable React to accurately determine which components need to be updated when the state changes

Creation of table in React

Table.jsx:-

```
import React from 'react'

const Table = () => {
  const Students = [
    { id: 1, name: "A", course: "Javascript" },
    { id: 2, name: "B", course: "Python" }
  ]

  return (
    <>
      <table>
        <thead>
          <tr>
            <th> ID </th>
            <th> NAME </th>
            <th> COURSE </th>
          </tr>
        </thead>
        <tbody>
          {StudentData.map((item) => (
            <tr key={item.id}>
              <td> {item.name} </td>
              <td> {item.course} </td>
            </tr>
          ))}
        </tbody>
      </table>
    </>
  )
}

export default Table
```


React life cycle:-

4 stages →
Initialising
Mounting
Updating
Unmounting

Class based Component :-

render
ComponentDidMount()
ComponentDidUpdate()
ComponentWillUnmount()

functional Component

In functional component, we use useEffect hook to work same as the lifecycle methods.
useEffect used to handle the side effects in functional component

Syntax:-

```
useEffect(() => {  
  first      → side effect function  
  return = () => { } → componentWillUnmount  
  second    → cleanup function  
}, [third])  
           ↳ dependencies
```

Case 1:- with no dependency

First function will run on every render

Ex:-

```
import React, { useState, useEffect } from 'react'
```

```
const CountUseEffect = () => {
```

```
  const [count, setCount] = useState(0);
```

```
  useEffect(() => {
```

```
    alert("running without dependencies")
```

```
  }, )
```

```
  function increment() {
```

```
    setCount(count + 1);
```

```
  }
```

```
  return (
```

```
    <div>
```

```
      <h1> Count: {count} </h1>
```

```
      <button onClick={increment}> increment </button>
```

```
    </div>
```

```
  )
```

```
  export default CountUseEffect
```

In the above example, the first function will run on every render, means we will get alert pop when the component mounts ^{for 1st time} and after clicking every button it keeps on running on every click.

Case 2:- Empty dependency is given

→ function will run only once

⇒ ComponentDidMount()

Case 3:- dependency is given

- function will run on every update of that dependency
- `ComponentDidUpdate()`

Case 4:- multiple dependencies

Whenever there is change of those dependencies then function will run

→ `ComponentDidUpdate()`

How to fetch the data from API

Steps:-

Step 1:- `fetch('url')`

Step 2:- Convert the data in `.json()`

Step 3:- Save the data in variable

Step 4:- Append the data

fetch.jsx

import React from 'react';

function fetch() {

const [users, setUsers] = useState([]);

useEffect(() => {

fetch('url').then((res) => res.json()).

.then((data) => setUsers(data));

}, [])[

return ({users.map((user) => {

return {user.name}

Case 3:- dependency is given

- function will run on every update of that dependency
- `componentDidUpdate()`

Case 4:- multiple dependencies

Whenever there is change of those dependencies then function will run

- `ComponentDidUpdate()`

How to fetch the data from API

Steps:-

Step 1:- `fetch('url')`

Step 2:- Convert the data in `.json()`

Step 3:- Save the data in variable

Step 4:- Append the data

fetch.jsx

import React from 'react';

function fetch() {

const [users, setUsers] = useState([]);

useEffect(() => {

fetch('url').then((res) => res.json()).

.then((data) => setUsers(data));

}, [])

return ({users.map((user) => {

return {user.name}

})

axios

It is a package of npm.
used for CRUD operations.

Routing/Routes

Step 1:- install react-router-dom

Step 2:- wrap your application with BrowserRouter

Step 3:- Define Routes Route

Step 4:- Use the routes → Link

App.jsx

```
import { BrowserRouter, Routes, Route } from "react-router-dom"
import Home from "../components/Home.jsx"
import About from "../components/About.jsx"
```

```
const App = () => {
```

```
  return (
```

```
    <div>
```

```
      <BrowserRouter>
```

```
        <Routes>
```

```
          <Route
```

path="/" element = {<Home/>} />

```
          <Route
```

path="/About" element = {<About/>} />

```
          <Route
```

path="*" element = {<pageNotFound/>} />

```
        </Routes>
```

```
      </BrowserRouter>
```

```
    </div>
```

```
  )
```

```
  export default App
```

↓
When the user clicks
for other Router it
will display the component
like pageNotFound

Navbar.jsx:-

```
import React from 'react'
import { Link } from 'react-router-dom'

function Navbar() {
  return (
    <Link to = '/'> Home </Link>
    <Link to = '/About'> About </Link>
  )
}
```

useNavigate

→ The `useNavigate()` hook in React Router v6 allows developers to programmatically navigate to different routes within a React application.

→ It returns a function, commonly named `navigate`, which can be used to change the current location, similar to how `<Link>` components work.

Syntax:-

```
const navigate = useNavigate();
navigate('/targetpath')
navigate('/path', { replace: true });
```

To navigate back to previous pages we will use the React `useNavigation` Hook,

// use this to go back to previous page
`navigate(-1)`

Ex:- App.jsx:-

```
import React from 'react';
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import Home from './components/Home';
import About from './components/About';
```



```
function App() {
```

```
  return (
```

```
    <>
```

```
      <BrowserRouter>
```

```
        <Routes>
```

```
          <Route path="/" element={ <Home /> } />
```

```
          <Route path="/about" element={ <About /> } />
```

```
        </Routes>
```

```
      </BrowserRouter>
```

```
    )
```

```
  };
```

Home.jsx:- import React from 'react';
import {useNavigate} from 'react-router-dom';

```
const Home = () => {
```

```
  const navigate = useNavigate();
```

```
  return (
```

```
    <>
```

```
      <h1 style={{color: "green"}}>POOJA HEDE </h1>
```

```
      <button onClick={() => navigate("/about")} >
```

```
        About
```

```
      </button>
```

```
    </>
```

```
  );
```

```
};
```

About.jsx:- import React from 'react';
import {useNavigate} from 'react-router-dom';
const About = () => {

```
  const navigate = useNavigate();
```

```
  return (
```

```
    <>
```

```
      <h1 style={{color: "green"}}>
```

```
        A Computer Science portal
```

```
      </h1>
```

```
      <button onClick={() => navigate(-1)} >
```

```
        Go Back Home
```

```
      </button>
```

```
    </>
```

```
  );
```

```
};
```


Controlled and Uncontrolled forms

Controlled forms

→ managed by
useState

→ return (

label

input → onChange → handleChange

button → onClick →
handleClick

↓
setName(e.target.value)

Uncontrolled forms

→ useRef

↓

It is used to
manipulate the DOM
/ it will give the
reference of DOM

import {useRef} from
'react'

const nameRef = useRef();

Uncontrolled forms

const nameRef = useRef()

variable

↓
const

nameRef = {

will have ← current;
the access of
DOM

=> nameRef.current.value

Ex - Form.jsx

```
import React from 'react'  
import {useRef} from 'react';
```

```
const form = () => {
```

```
  const nameRef = useRef()
```

```
  function handleClick() {
```

```
    console.log(nameRef.current.value)
```

```
  }
```

```
  return (
```

```
    <label>Firstname </label>
```

```
    <input type="text" placeholder="Enter the first name"  
      ref={nameRef} />
```

```
    <button onClick={handleClick}>Submit </button>
```

```
  )  
  export default form
```


Controlled forms

```
import React, {useState} from "react";  
const Controlled = () => {  
  const {name, setName} = useState("");  
  function handle(e) {  
    setName(e.target.value);  
  }  
}
```

```
function handleClick(e) {  
  e.preventDefault();  
  console.log(name);  
  setName("");  
}
```

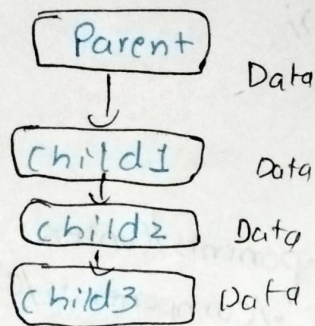
```
return (  
  <div>  
    <form onSubmit={handleSubmit}>  
      <label htmlFor="nameInput" className="font-bold">  
        Enter the name: </label>  
      <input id="nameInput" type="text" placeholder="Enter the name" />  
      <button type="submit"> Submit </button>  
    </form>  
  </div>  
)  
};
```


NOTE:-

- Controlled Components use React state to manage input values
- Uncontrolled Components rely on the DOM to handle the data

Use Context :- Context API

Store the data at one place, and whichever child want to access the data. They can use



↓ unidirectional

↓
Props drilling

↓
To pass props unnecessarily through multiple intermediate components, making the code ~~messy~~ and difficult to manage

React Context is a way to manage state globally

Steps:-

- Step 1:- createContext
- Step 2:- provide the Context-Provider (wrap your application)
- Step 3:- consume the Context (useContext)

File

```
const userContext = createContext()
```

App.jsx

```
const user = {name: "Pooja"}  
return C
```

```
<useContext.Provider>
```

```
  <store the data here>
```

```
</useContext.Provider>
```

Example:-

Components/Context.jsx :

```
import {createContext} from 'react';
```

```
const Context = createContext();
```

```
export default Context;
```

App.jsx:-

```
import Context from './components/context'
```

```
import ChildComponent from './Components/childComponent'
```

```
function App() {
```

```
  const Student = {name: "Pooja"};
```

```
  return C
```

```
    <>
```

```
    <Context.Provider value={Student}>
```

```
      <ChildComponent />
```

```
    </Context.Provider>
```

```
  </>
```

```
}
```

```
export default App;
```


components/ChildComponent.jsx

```
import React, { useContext } from 'react'
import Context from './Context';

const ChildComponent = () => {
  const ConsumeContext = useContext(Context);
  return (
    <div><h1> {ConsumeContext.name} </h1></div>
  )
}

export default ChildComponent
```

Custom Hooks:-

By using default hooks we will create our own hook

Benefits:-

- Reduce your code
- reusability increases
- faster

Custom hook in react is a user-defined function that starts with the word use and allows you to reuse stateful logic across multiple components. It keeps the code clean, and avoid repeating of the same logic.

Eg:- For Counter:-

useCount.jsx:-

```
import React, { useState } from 'react'
const [Count, setCount] = useState(0)
function Increment() {
  setCount(count + 1)
}
function decrement() {
  setCount(count - 1)
}
```



```
return (  
  [count, Increment, decrement]  
)  
export default useCount;
```

Count.jsx:-

```
import useCount from './useCount'  
const Count = () => {  
  const [count, increment, decrement] = useCount(0);  
  return (  
    <>  
    <h> {count} </h>  
    <button onClick = {Increment}> Add </button>  
    <button onClick = {decrement}> decrement </button>  
    </>  
  )  
}
```

H.W:- Fetch an API using Custom hook

Lazy Loading:

It is used for better user experience

Step1:- React.lazy whatever component you want to apply lazy thing, you will use lazy

→ import inside lazy

Step2:- Suspense: Wrap your component inside suspense

Step3:- fallback: It is passed inside Suspense
→ whatever content present inside fallback will be append on the screen (loading...)

Lazy Loading:- Lazy loading in react is a techniques used to delay the loading of a component until it is actually needed.

Foreg:- When user navigates to specific route

- This improves the initial load time & performance of the application by splitting the code into smaller chunks
- * `Read.lazy()`: is used to dynamically import a component
 - * `Suspense`: is used to wrap lazy-loaded components and display a fallback UI (like a loading message or spinner) while those components are being loaded

fallback:- It's a special prop of the `Suspense` component that defines what should be displayed while a lazy-loaded component is still loading

Lazy loading:-

Eg:- lazy.jsx

```
import { Suspense } from 'react'
const load = React.lazy(() => import('./Navbar/
nav.jsx'))
```

```
return (
```

```
<>
  <Suspense fallback={<h3>loading...</h3>}>
```

```
    <Load>
```

```
  </Suspense>
```

```
</>
```


useMemo:

It will memoize / memorize the value of
Expensive Operation (the operation which take some time)

→ mainly used for optimization

Person 1	useMemo value	Person 2
$16 \times 8 = 128$		128
$128 \times 8 = 1024$		1024
$1024 \times 9 = 9216$		9216

In case of useCallback it will memoize the function itself.

Ex:-

optimization.jsx :-

```
import { useState, useMemo } from 'react';
```

```
const Optimization = () => {  
  const [Input, setInput] = useState(0);
```

```
  function expensiveTask(num) {  
    console.log("Inside the expensive Task");  
    for (let i = 0; i < 10000000; i++) {}  
    return num * 2;  
  }
```



```
let doubleValue = useMemo(() => expensiveTask(input), [input])
```

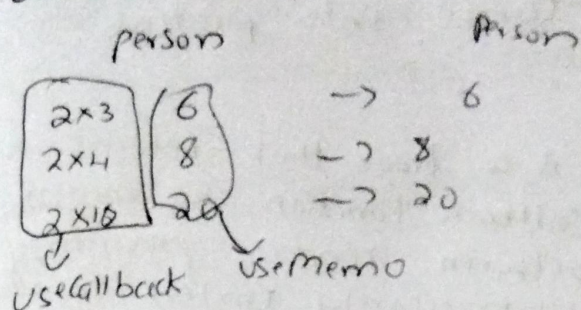
```
return (
```

```
  <input type="number" onChange={e => {setInput(e.target.value)}}
```

```
  <h2> {doubleValue} </h2>
```


useCallback :-

It freeze the function, and memorize the function expression



React. Memo :-

We use this method of memoization to save the function and prevent the child component to re-render unnecessarily when we passing props like text

Ex:- Callback.jsx

import React from 'react'

```
const Callback = React.memo ((props) => {  
  console.log ("callback is running")
```

```
  return C
```

```
    <div> <button> {props.name} </button> </div>
```

```
  )  
}
```

```
export default Callback;
```

App.jsx :-

```
import Callback from './components/callback'  
function App() {
```

```
  const [count, setCount] = useState(0);
```

```
  return C
```

```
    <>  
    <h3> Count: {count} </h3>
```

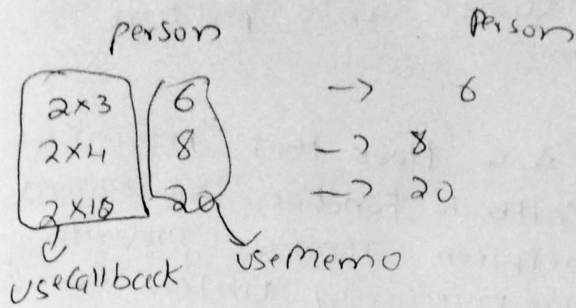
```
    <button onClick={ () => setCount(count+1)} >
```

```
      <del>callback name</del> - Increment </button>
```

```
    </>  
    <Callback name="Pooja" />  
  } </>
```


useCallback :-

It freeze the function, and memorize the function expression



React. Memo :-

We use this method of memoization to save the function and prevent the child component to re-render unnecessarily when we passing props like text

Ex:- Callback.jsx

```
import React from 'react'
```

```
const Callback = React.memo (Cprops) => {  
  console.log ("callback is running")
```

```
  return C
```

```
  <div> <button> {props.name} </button> </div>
```

```
  }  
}
```

```
export default Callback;
```

-App.jsx:-

```
import Callback from './components/callback'  
function App() {
```

```
  const [count, setCount] = useState(0);
```

```
  return C
```

```
    <>  
    <h3> Count: {count} </h3>
```

```
    <button onClick={ () => setCount(count+1)} >
```

```
      <del>callback name</del> - Increment </button>
```

```
    </>  
    <del>callback name</del> = "Pooja" </>  
  }  
</>
```


Now, if I want to pass the function as a prop then React.memo can't handle this so, the useCallback came into picture

useCallback

useCallback in React is a Hook that returns a memoized version of a callback function. This means it caches the function between renders, preventing it from being recreated unnecessarily unless one of its dependencies changes.

Ex:-

App.jsx:-

```
import Callback from './components/Callback'  
function App() {
```

```
  const [count, setCount] = useState(0);
```

```
  const handleClick = useCallback (() => {  
    setCount(count + 1);  
  }, []);
```

```
  return (
```

```
    <>
```

```
    <h3>Count: {count}</h3>
```

```
    <button onClick = {() => setCount(count + 1)}>  
      Increment </button>
```

```
    <Callback name = "Pooja" handleClick = {handleClick} />
```

```
  </>
```

```
)
```

```
}
```


Callback.jsx :-

```
import React from 'react'
const Callback = React.memo((props) => {
  console.log ("Callback is running")
  return (
    <div> <button onClick={props.handleClick}>
      {props.name}</button>
    </div>
  )
})
export default Callback
```

This hook improve performance, especially when the function is passed as a prop to a child component that is optimized with React.memo

When to use `useNavigate()` vs `<Link>`

`useNavigate()`: Ideal for programmatic navigation, such as after form submissions, authentication or other events, where navigation is triggered by code rather than user interaction

`<Link>` or `<NavLink>`: Best for declarative navigation, where the navigation is part of the UI and directly tied to user interaction like menu items or navigation bars